CS 650 – Full Stack Application Develop

2-2 Activity: Mapping Your API

Malgorzata Debska

Southern New Hampshire University

Brian Enochson

July 14, 2026

Fernbase API Routing Activity

API Routing Plan

The Fernbase application provides users with the ability to browse, review, and rate houseplants. The back-end API is responsible for handling requests from the client application and returning JSON responses. The API follows REST principles by using different HTTP methods to perform CRUD (Create, Read, Update, Delete) operations on plant resources. In addition, ratings are implemented as a nested resource because every rating belongs to a specific plant. The routes are implemented using Express and TypeScript, while the database operations are handled through the provided service layer.

HTTP Method	Route	Query Parameters / Request Body	Purpose	Response
GET	/plants	Optional query parameter: careLevel	Returns all plants or filters plants by care level.	200 OK with JSON array of plants
GET	/plants/:id	Path parameter: id	Retrieves detailed information for a single plant.	200 OK or 404 Not Found
POST	/plants	JSON body containing plant information	Creates a new plant record.	201 Created or 400 Bad Request
PUT	/plants/:id	Path parameter and JSON body	Updates an existing plant.	200 OK, 400 Bad Request, or 404 Not Found
DELETE	/plants/:id	Path parameter	Removes a plant from the collection.	200 OK or 404 Not Found
GET	/plants/:id/ratings	Path parameter	Returns all ratings for a specific plant.	200 OK or 404 Not Found

POST	/plants/:id/ratings	Path parameter and JSON body	Adds a new rating for a plant.	201 Created, 400 Bad Request, or 404 Not Found

Route Documentation

1. List All Plants

HTTP Method: GET

Route: /plants

Purpose

This endpoint returns every plant stored in the Fernbase application. An optional query parameter allows users to filter the results by care level, making it easier to locate plants that match a specific experience level.

Sample Requests

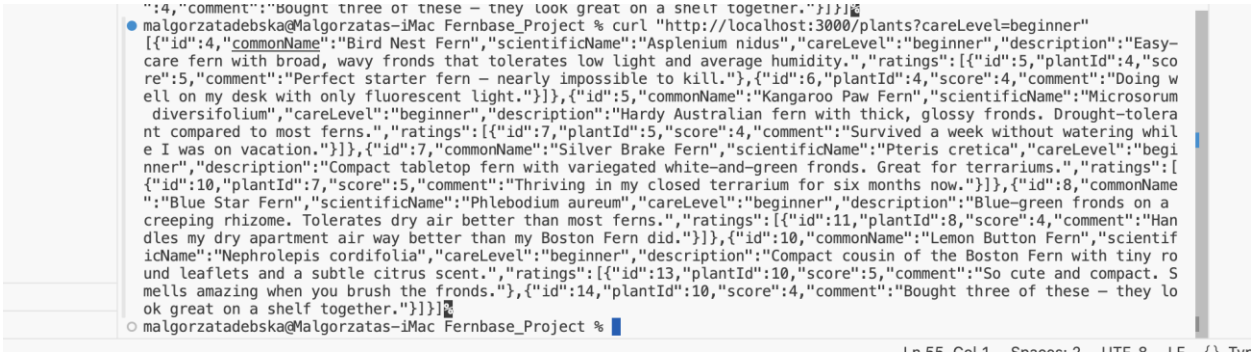
curl http://localhost:3000/plants

Filter by care level:

curl "http://localhost:3000/plants?careLevel=beginner"

Expected Response

- 200 OK – Returns a JSON array containing all plants or only plants that match the selected care level.



2. Retrieve a Single Plant

HTTP Method: GET

Route: /plants/:id

Purpose

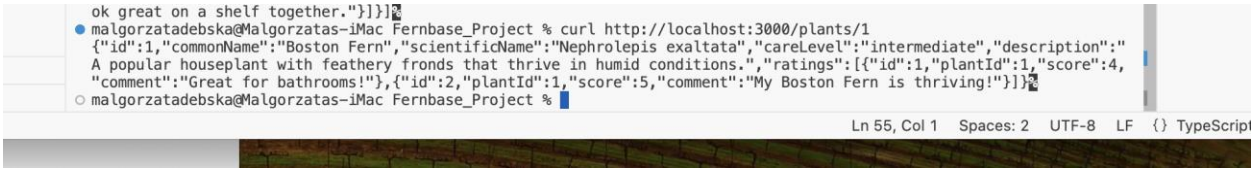
This endpoint retrieves detailed information about a specific plant using its unique identifier. The response includes the plant's common name, scientific name, care level, description, and any ratings associated with it.

Sample Request

curl http://localhost:3000/plants/1

Expected Response

- 200 OK – Returns the requested plant as a JSON object.
- 404 Not Found – Returned if the specified plant does not exist.



3. Add a New Plant

HTTP Method: POST

Route: /plants

Purpose

This endpoint allows users to add a new plant to the Fernbase collection. The request body contains the plant information in JSON format, and the API validates the data before creating the record.

Sample Request

curl -X POST http://localhost:3000/plants \

-H "Content-Type: application/json" \

-d '{"commonName":"Button Fern","scientificName":"Pellaea rotundifolia","careLevel":"beginner","description":"A compact fern."}'

Expected Response

- 201 Created – Returns the newly created plant with its generated ID.
- 400 Bad Request – Returned when required fields are missing or invalid.

4. Update a Plant

HTTP Method: PUT

Route: /plants/:id

Purpose

This endpoint updates the information for an existing plant. The client sends the updated data as JSON, and the API replaces the existing values with the new information.

Sample Request

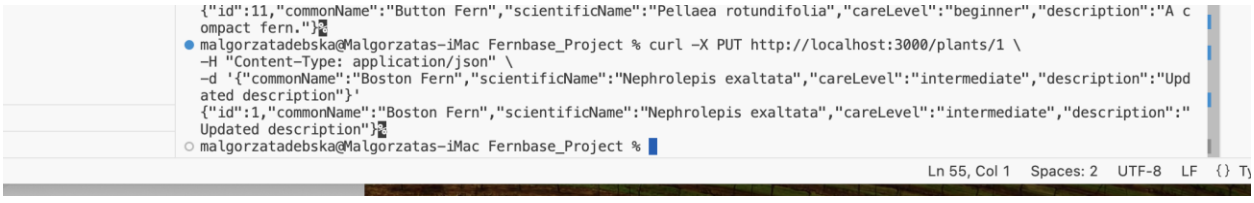
curl -X PUT http://localhost:3000/plants/1 \

-H "Content-Type: application/json" \

-d '{"commonName":"Boston Fern","scientificName":"Nephrolepis exaltata","careLevel":"intermediate","description":"Updated description"}'

Expected Response

- 200 OK – Returns the updated plant.
- 400 Bad Request – Invalid request body.
- 404 Not Found – Plant ID does not exist.



5. Remove a Plant

HTTP Method: DELETE

Route: /plants/:id

Purpose

This endpoint removes an existing plant from the database using its unique identifier.

Sample Request

curl -X DELETE http://localhost:3000/plants/11

Expected Response

- 200 OK – Returns a confirmation message indicating that the plant was successfully removed.
- 404 Not Found – Plant could not be found.

6. View Plant Ratings

HTTP Method: GET

Route: /plants/:id/ratings

Purpose

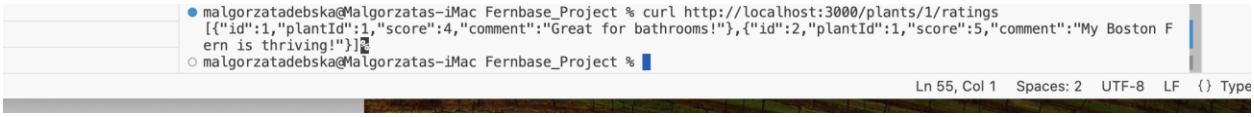
This endpoint returns every rating and comment associated with a specific plant. Ratings are treated as nested resources because they belong to an individual plant.

Sample Request

curl http://localhost:3000/plants/1/ratings

Expected Response

- 200 OK – Returns a JSON array containing all ratings for the selected plant.
- 404 Not Found – Plant ID does not exist.



7. Add a Rating

HTTP Method: POST

Route: /plants/:id/ratings

Purpose

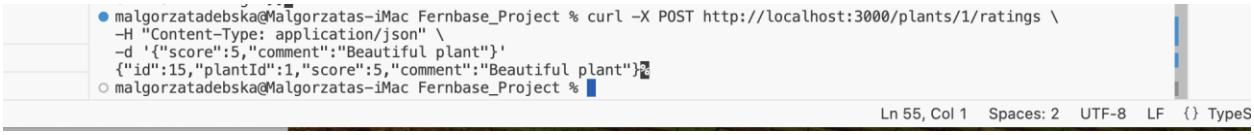
This endpoint allows users to submit a rating for a plant. The request includes a score between 1 and 5 and may also include an optional written comment.

Sample Request

```
curl -X POST http://localhost:3000/plants/1/ratings \
-H "Content-Type: application/json" \
-d '{"score":5,"comment":"Beautiful plant"}'
```

Expected Response

- 201 Created – Returns the newly created rating.
- 400 Bad Request – Invalid rating information.
- 404 Not Found – Plant ID does not exist.



Route Configuration Summary

The Fernbase API was designed using RESTful principles to provide a clear and consistent structure for managing plant data and ratings. Collection routes (/plants) are used to retrieve or create plant records, while individual resource routes (/plants/:id) are used to retrieve, update, or delete a specific plant. Ratings are implemented as nested resources (plants/:id/ratings) because each rating belongs to a single plant.

Each route returns JSON responses and appropriate HTTP status codes, making the API predictable and easy to integrate with a front-end application. This design also improves maintainability by separating responsibilities between routing, validation, and the service layer.

Overall, the routing structure supports the application's core functionality while following common software engineering practices for REST API development.